

Automated Docker Deployment to AWS EC2 Using GitHub Actions CI/CD

Objective: To implement a CI/CD pipeline using **GitHub Actions** that automatically triggers on code changes, builds a Docker image, and deploys the application to an **AWS EC2** instance.

Prerequisites

1. GitHub Secrets: I configure the following secrets in the GitHub repository settings:

DOCKER_USERNAME – Docker Hub username

DOCKER_PASSWORD – Docker Hub password or access token

EC2_HOST – Public IP address of the EC2 instance

EC2_SSH_KEY – Full content of the EC2 private key (.pem file)

2. EC2 Instance Setup: Install Docker on the EC2 instance & Grant Docker permissions to the user: `sudo usermod -aG docker $USER`

3. Application Code: app.js, package.json, Dockerfile

4. GitHub Actions Workflow: Create the workflow file at- .github/workflows/deploy.yml

This workflow automates the build and deployment process whenever code is pushed to the repository.

System Architecture: The workflow follows a linear four-step process:

- i. **Code Commit:** pushes code to the main branch on GitHub.
- ii. **Continuous Integration:** GitHub Actions triggers a runner to build the Docker image.
- iii. **Image Registry:** The authenticated runner pushes the tagged image to **Docker Hub**.
- iv. **Continuous Deployment:** GitHub Actions connects to the **AWS EC2** instance via SSH, pulls the latest image, and restarts the container.

POC:

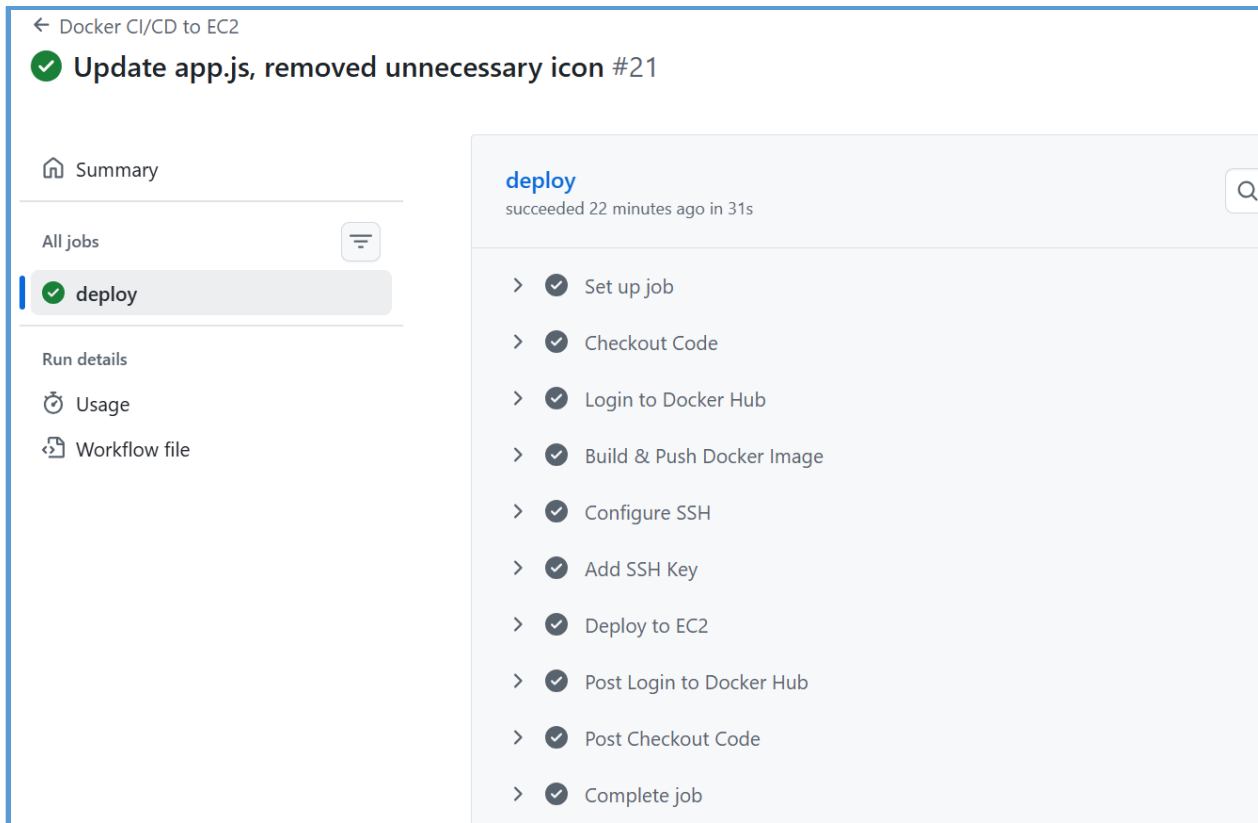


Fig. 1: Successful deployment message displayed in GitHub Actions

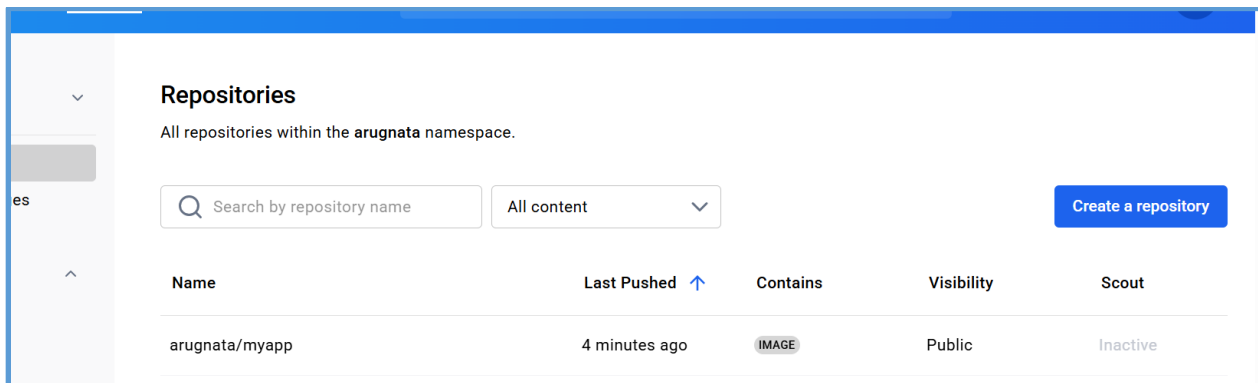


Fig. 2: Docker image successfully pushed to Docker Hub

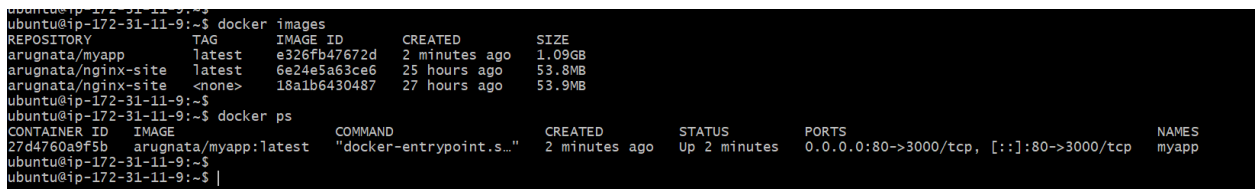


Fig. 3: Docker image deployed on AWS EC2 with the container running

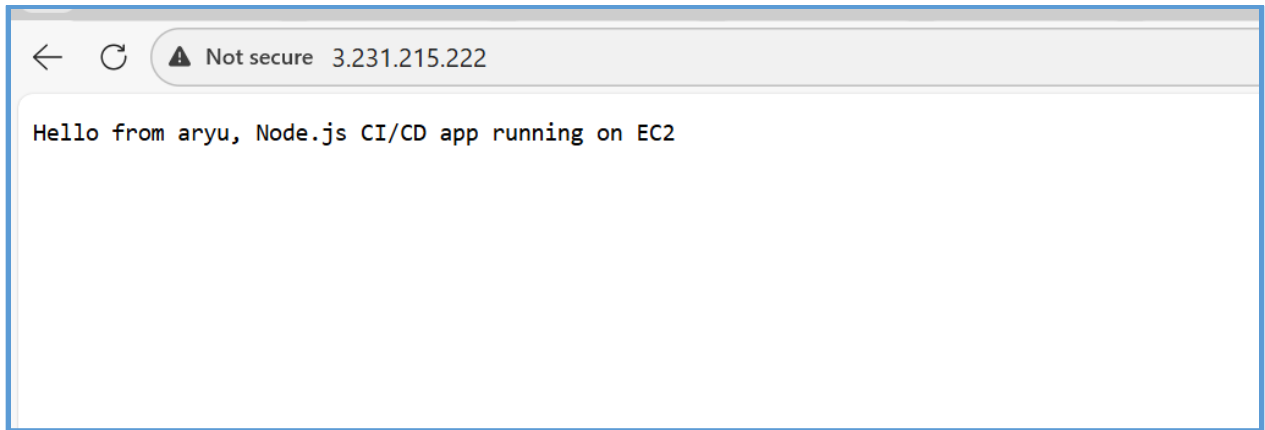


Fig. 4: Application running and accessible via the assigned port